



DS211-Theory of Data Science

Spring 2026

Lecture 10 – Text Data

Dr. Shahab Ansari

sansari@giki.edu.pk

String Data

- There are four kinds of string data you might see:
 - Categorical data
 - Free strings that can be semantically mapped to categories
 - Structured string data
 - Text data

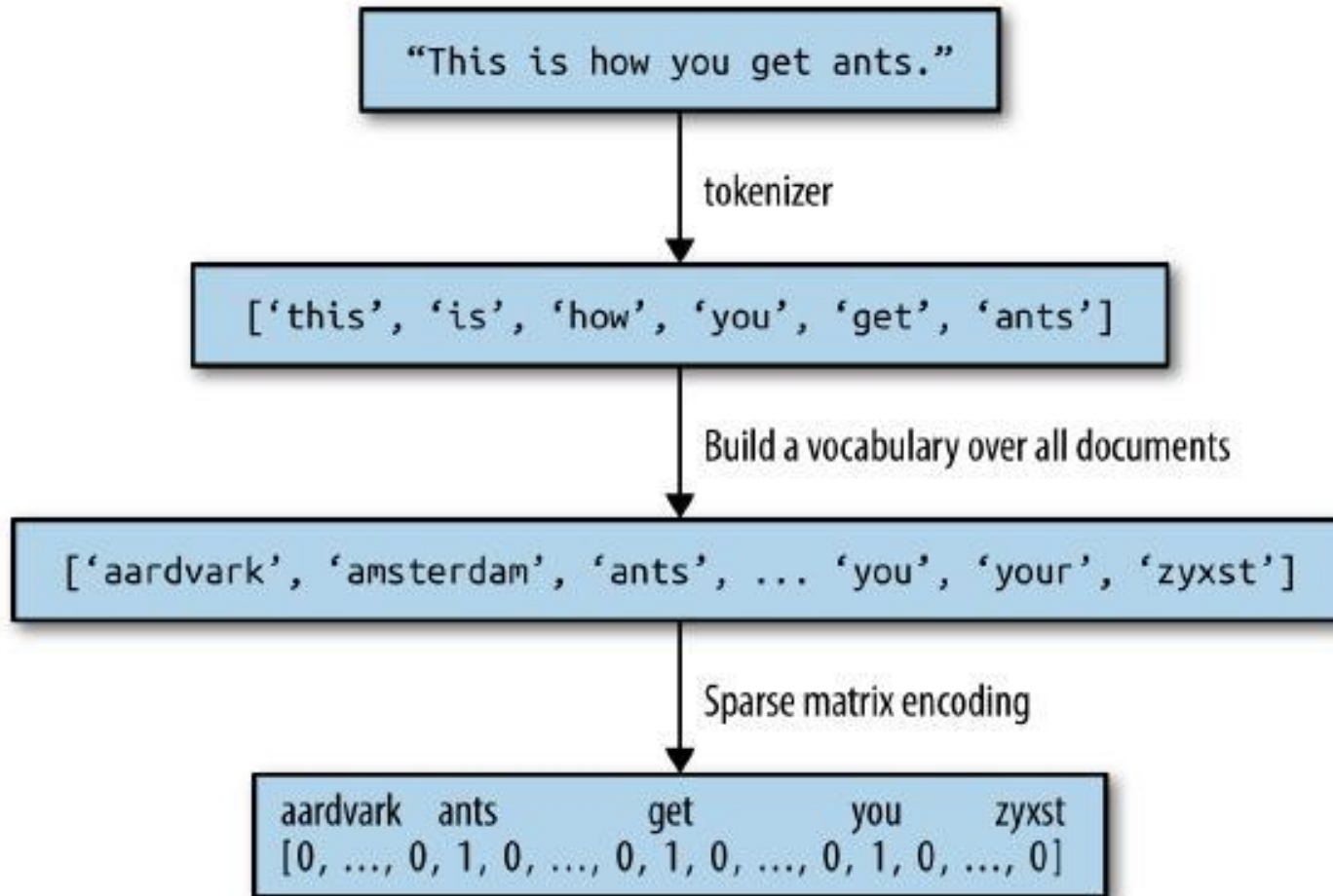
Text Data

- One of the most simple but effective and commonly used ways to represent text for machine learning is using the **bag-of-words** representation.
- When using this representation, we **discard most of the structure** of the input text and only count how often each word appears in each text in the corpus.

Text Data

- Computing the bag-of-words representation for a corpus of documents consists of the following three steps:
 - Tokenization. Split each document into the words that appear in it (called tokens), for example by splitting them on whitespace and punctuation.
 - Vocabulary building. Collect a vocabulary of all words that appear in any of the documents, and number them (say, in alphabetical order).
 - Encoding. For each document, count how often each of the words in the vocabulary appear in this document.

Text Data



Text Data

- Stopwords
 - Stopwords are the uninformative words that are too frequent to be informative.
 - Removing the stopwords in the list can only decrease the number of features but it might lead to an improvement in performance and interpretability.

Rescaling the Data

- Instead of dropping features that are deemed unimportant, another approach is to rescale features by how informative we expect them to be.
- One of the most common ways to do this is using the term frequency–inverse document frequency (tf–idf) method.
- The intuition of this method is to give high weight to any term that appears often in a particular document, but not in many documents in the corpus.
- If a word appears often in a particular document, but not in very many documents, it is likely to be very descriptive of the content of that document.

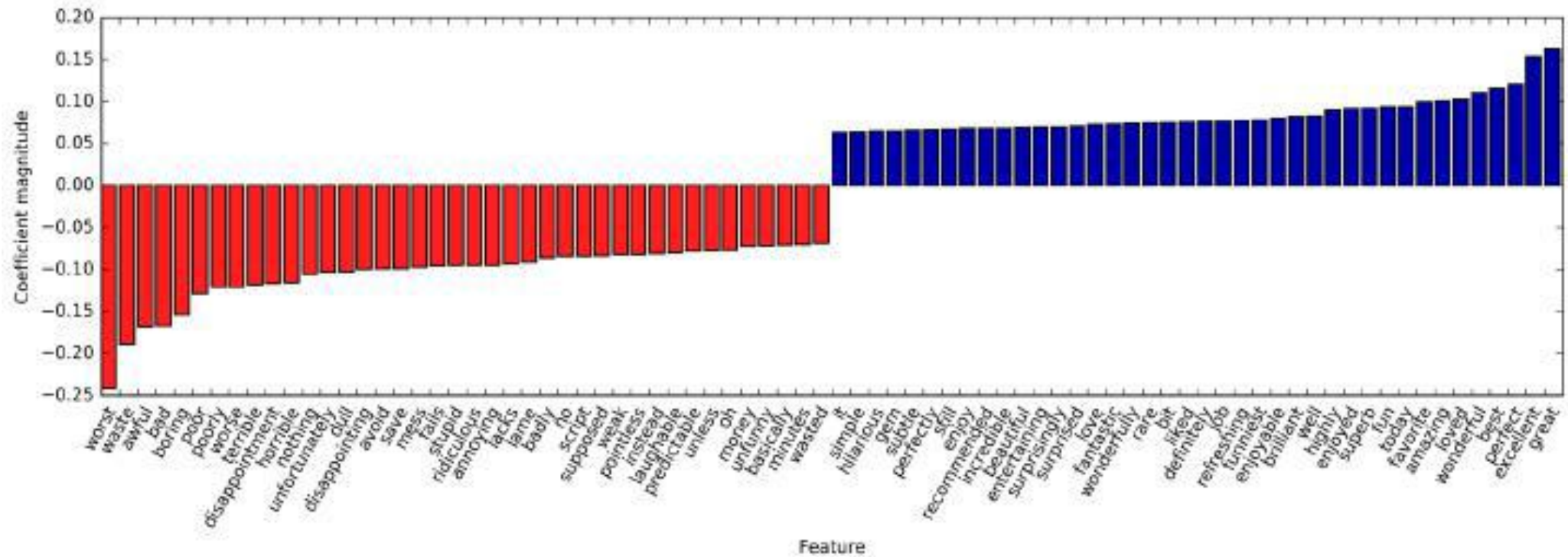
Rescaling the Data

- The tf-idf score for word w in document d is given by:

$$tfidf = tf \cdot \log \left(\frac{N + 1}{N_w + 1} \right) + 1$$

- where N is the number of documents in the training set, N_w is the number of documents in the training set that the word w appears in, and tf (the term frequency) is the number of times that the word w appears in the query document d (the document you want to transform or encode).

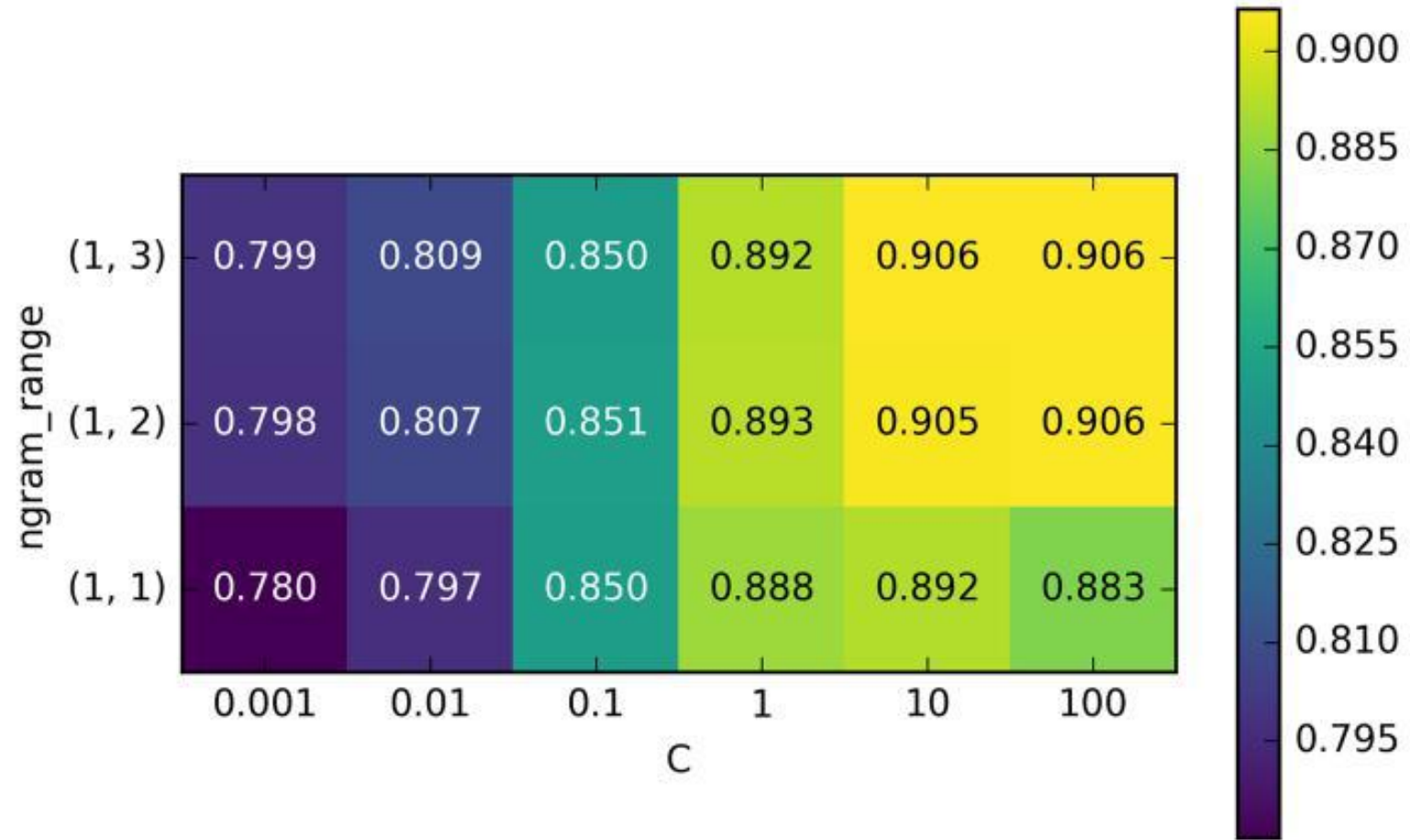
Rescaling the Data



N-grams

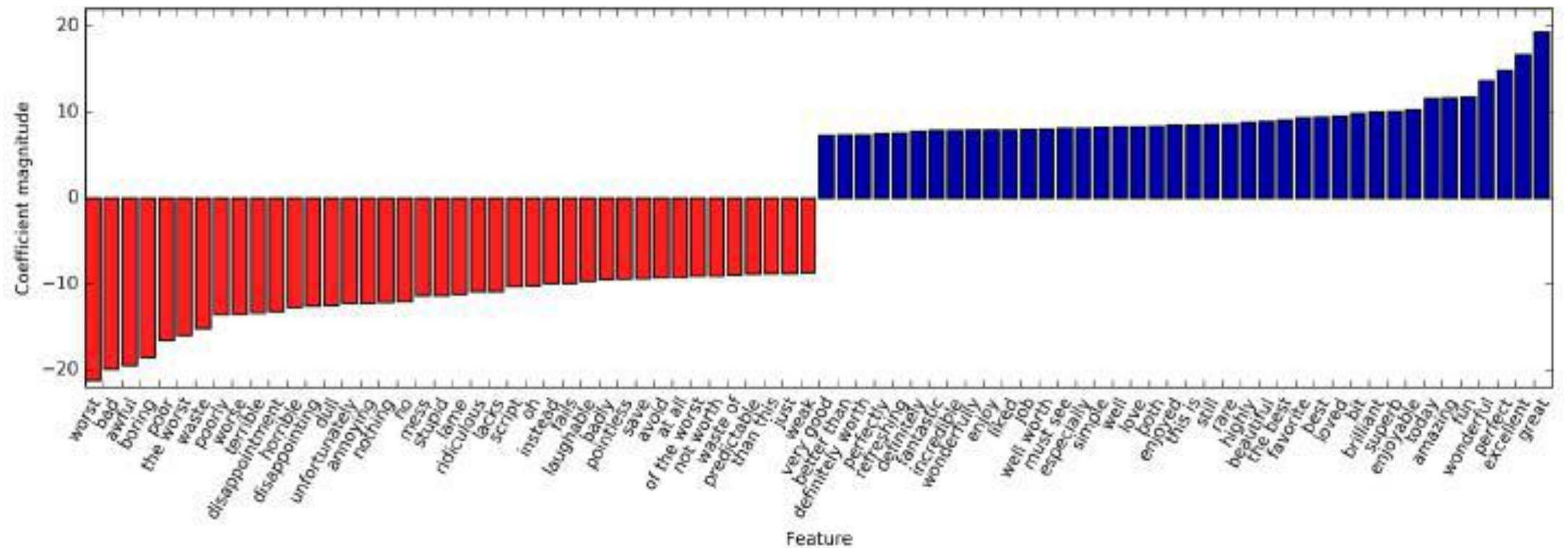
- One of the main disadvantages of using a bag-of-words representation is that word order is completely discarded.
- Fortunately, there is a way of capturing context when using a bag-of-words representation, by not only considering the counts of single tokens, but also the counts of pairs or triplets of tokens that appear next to each other.
- Pairs of tokens are known as bigrams, triplets of tokens are known as trigrams, and more generally sequences of tokens are known as n-grams.

N-grams



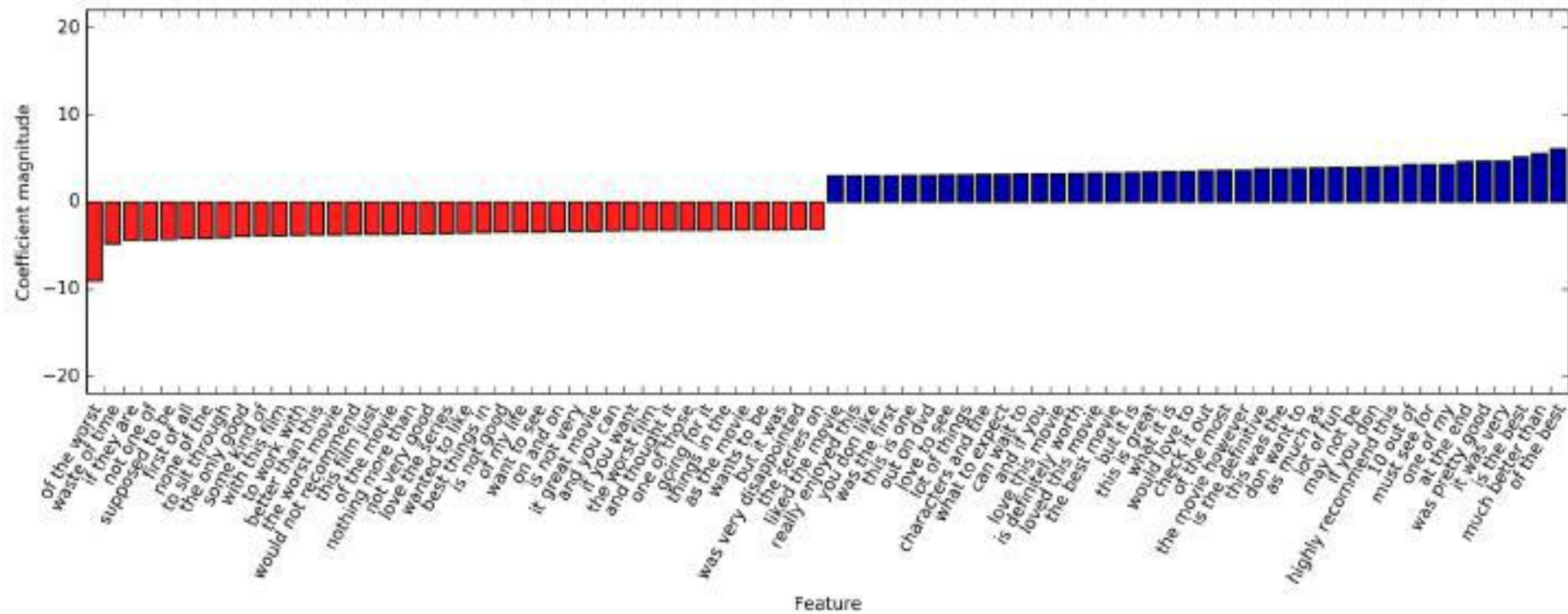
N-grams

Important unigram and bigram features



N-grams

Important trigram features



Tokenization

- The feature extraction in the CountVectorizer and Tfidf Vectorizer is relatively simple.
- Much more elaborate methods are possible.
- One particular step that is often improved in more sophisticated text-processing applications is the **tokenization**.
- This step defines what constitutes a word for the purpose of feature extraction.

Tokenization

- We can define the following rules for tokenizing the sentence using regular expressions:
 - Words do not contain spaces. In this sense, we can split the sentence into “word candidates” with space.
 - Every word candidate that is made up of English letters only (i.e., a-z and A-Z) is a word.
 - Every punctuation mark (i.e., quote, comma, period, etc.) should be isolated to form a word.
 - 's is a word, indicating noun possessive.

Text Normalization

- Lemmatization
 - Also, it is common to normalize the text before tokenization so that the input of the tokenizer is canonical.
 - For example, normalization or canonicalization refers to a process of lowercasing words, normalizing character representation (e.g., Unicode characters), mapping different forms of a word to the same form for further generalization of the tokenization.
 - A simple way to do this is to conflate all inflected forms of a word into its base form.
 - In linguistics, the base form of a word is called lemma, and the process of mapping words to lemmas is called lemmatization (context dependent).
 - learn → learn
 - learning → learn
 - learns → learn
 - best → good

Text Normalization

- Stemming
 - Closely related to lemmatization is stemming, which represents a word as its stem.
 - Like lemmas, a stem is some base form of a word.
 - However, unlike lemmas, a stem is not necessarily a valid word, although there are many words whose lemmas and stems are identical.
 - Another difference from lemmatization is that stemming is performed on individual words, without the need of context for disambiguation.
 - So, stemming is context-independent.
 - There are several efficient algorithms for stemming (such as suffix stripping).
 - remove → remov
 - removing → remov
 - removal → remov
 - best → best

Vectorization

- One-Hot Coding
- Count-Based Encoding
- Model-Based Encoder

Vector Representation for Words

- One-Hot Coding
 - To represent a word, the simplest idea may be to replace it with the id number in the dictionary.
 - In this way, each word representation is a unique number.
 - An equivalent form to this is the one-hot representation.
 - It is a vector whose dimensionality is equal to the vocabulary size.
 - In this vector, only the entry corresponding to the word has a value of 1 and all other entries have 0 values.
 - An obvious problem is that every word is **orthogonal** to other words.
 - This makes it difficult to “compute” the relationship between words.

Vector Representation for Words

- Count-Based Encoding
 - Here, our desire is a model in which words are described as countable attributes and the closeness between different words is well explained.
 - This way enables the sharing of content among words.
 - We would say that two words are similar if they have overlaps in their word vectors.
 - A simple improvement to one-hot encoding is to use a function counting the occurrences of a word.

Vector Representation for Words

- Model-Based Encoder (Representation Learning)
 - A more general form for word representation is simply a real-valued, multi-dimensional vector often called the **distributed representation of a word**, or the **word embedding**.
 - The relationship between words is often thought of as some “distance” between the word representations in a vector space.
 - This leads to a number of methods to visualize and evaluate word representations.
 - The representation model works as a component of a **big system**, and is optimized together with other components when the system is trained in some way.

Thank You!

sansari@giki.edu.pk